

CMPT 361 Report

Spring 2024

Instructor: Mohammad Elmorsy

Shadr Baaba, Rimneet Cheema, Jamie McDonald

Index

Project Details	3
Testing Server.py and Client.py	6
Attack	9
Enhanced protocol	9
Testing Server_enhanced.py and Client_enhanced.py	10
Resources.....	11,12

Project Details

Programs and Status:

- Key_generator.py (Complete)
 - This program will generate the RSA key pairs for the clients and server and save them accordingly.
 - Imports
 - from Crypto.PublicKey import RSA
 - Import os
 - Generates 2048-bit key pairs
 - Saves server public both in server folder and client folder
 - Saves server private to only server folder
 - Saves clients key pairs to their specific folder, and saves copy of their public key in their corresponding folder in the server folder
 - Generates inboxes for clients (there is also functions in server.py to generate folders when sending an email or viewing inbox for good measures)
- client.py (Complete)
 - This program creates a client that can connect to a server and allow for secure emailing protocols between clients
 - Imports:
 - import socket
 - import os
 - import json
 - from Crypto.PublicKey import RSA
 - from Crypto.Cipher import PKCS1_OAEP, AES
 - from Crypto.Util.Padding import pad, unpad
 - This program initially will prompt users to input server name
 - Then they are prompted for their credentials and using servers public their username and password is RSA encrypted and sent to the server
 - After authentication the client will receive the symmetric key encrypted in their corresponding public key. Using their own private key they will encrypt the

symmetric key and use this for any further communication with the server (AES-ECB mode).

- The client will handle the main client operations such as sending emails, viewing inbox, viewing email content, and terminating connection.
- server.py (Complete)
 - This program creates a secure emailing server that allows authorized users to send emails, view inbox, and view emails.
 - Imports
 - import socket
 - import os
 - import json
 - import sys
 - from datetime import datetime
 - from Crypto.PublicKey import RSA
 - from Crypto.Cipher import PKCS1_OAEP, AES
 - from Crypto.Util.Padding import pad, unpad
 - from Crypto.Random import get_random_bytes
 - This program will start the server, listen for incoming client connections, when client connects it will create a child process to handle the client, it can have up to 5 clients connected at the same time.
 - Handling clients. The server when connecting with the client will receive their credentials which are encrypted with the server public key. Using the server private key, it will decrypt the credentials and check for authentication. If the user is not authorized the connection will be terminated. If the user passes authentication then the server will generate an AES symmetric key, encrypt it using the specific client's public key and send it to the client and wait to receive an "OK" message encrypted with the symmetric key.
 - The server will then do the main loop where it will send the menu to the client and will do whichever emailing protocol is selected by the client. All with secure AES encryption.
- Test_server.py
 - This program was useful early on but due to time constraints is incomplete.

Assumptions taken while developing the project:

- Predefined clients:
 - There is a fixed set of known clients. Their usernames, passwords and corresponding keys are all predefined either in the JSON file or generated by the key generator.
 - We assumed that this program would be developed using python and that it would be run in a Linux environment.
- Secure connection/Keys
 - Data will be multiples of 256 for AES encryption (and valid)
 - Symmetric key encryption will be in ECB
 - Authentication will be done with RSA key pairs for each client and server
 - Remaining communication will be encrypted through AES

Deficiencies in the developed programs:

- Known Bug
 - When viewing an empty inbox, the client side crashes, upon adding detailed debugging and testing the code, it was discovered the client side would crash due to receiving and attempting to decrypt an empty response. This happened when the server tried to send an "Inbox is empty." message but encountered issues during the encryption or transmission process. Despite implementing error handling measures to address this issue, a new problem occurred which made inboxes with emails inaccessible. This further complicated the issue and due to the limited time available for this project as well as the high importance for the functionality of viewing an inbox with emails, this issue dropped in priority.

Testing Server.py and Client.py

```
cheemar9@cc5-212-16:~/ProjectG3/server$ python3 server.py
The server is ready to accept connections
The received client information: client10 is invalid (Connection Terminated).
Connection Accepted and Symmetric Key Generated for client: client1
An email from client1 is sent to ['client2', 'client3'] has a content length of 20
Terminating connection with client1
Connection Accepted and Symmetric Key Generated for client: client3

cheemar9@cc5-212-11:~/ProjectG3/client$ python3 client.py
Enter server name: cc5-212-16.macewan.ca
Enter your username: client10
Enter your password: password10
Invalid username or password
Invalid username or password
cheemar9@cc5-212-11:~/ProjectG3/client$ python3 client.py
Enter server name: cc5-212-16.macewan.ca
Enter your username: client1
Enter your password: password1
Authentication successful.
```

Fig.1

```
cheemar9@cc5-212-16:~/ProjectG3/server$ python3 server.py
The server is ready to accept connections
The received client information: client1 is invalid (Connection Terminated).

cheemar9@cc5-212-11:~/ProjectG3/client$ python3 client.py
Enter server name: cc5-212-16.macewan.ca
Enter your username: client1
Enter your password: wrongpassword
Invalid username or password
```

Fig. 2

Test 1- Authentication:

- client10 attempts to connect (Fig. 1)
- Since client10 is not an authorized user the connection is terminated (Fig. 1)
- client1 attempts to connect (Fig. 1)
- Connection is accepted and Symmetric key is generated (Fig. 1)
- client1 attempts to connect with wrong password (Fig.2)
- Connection is terminated. (Fig. 2)

```
cheemar9@cc5-212-16:~/ProjectG3/server$ python3 server.py
The server is ready to accept connections
The received client information: client10 is invalid (Connection Terminated).
Connection Accepted and Symmetric Key Generated for client: client1
An email from client1 is sent to ['client2', 'client3'] has a content length of 20
Terminating connection with client1
Connection Accepted and Symmetric Key Generated for client: client3

cheemar9@cc5-212-11:~/ProjectG3/client$ python3 client.py
Enter server name: cc5-212-16.macewan.ca
Enter your username: client10
Enter your password: password10
Invalid username or password
Invalid username or password
cheemar9@cc5-212-11:~/ProjectG3/client$ python3 client.py
Enter server name: cc5-212-16.macewan.ca
Enter your username: client1
Enter your password: password1
Authentication successful.

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 1
Enter recipient(s) (separated by ;) : client2;client3
Enter title: ProjectTEST
Enter message contents: this is project test
Email sent successfully.

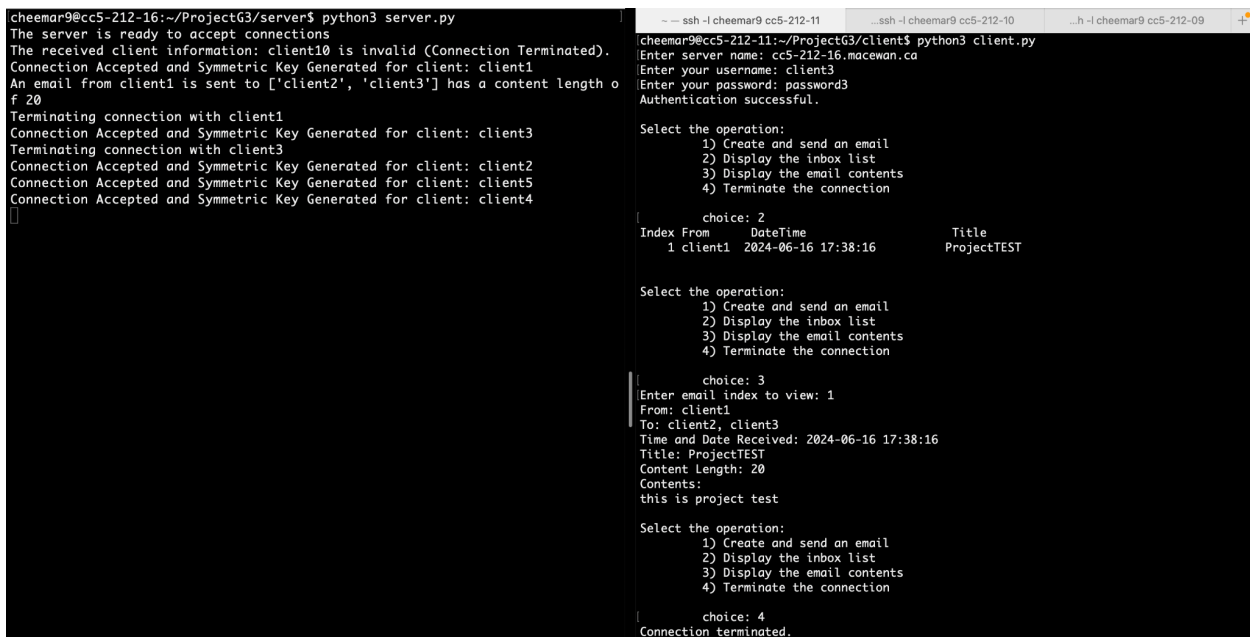
Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 4
Connection terminated.
```

Fig. 3

Test 2 - Send email and terminate (Fig. 3)

- Menu is presented to client1
- Client1 selects option 1 (send email)
- Client1 is given prompt for recipients client1 choose client2; client3
- Server prints email confirmation statement
- Client1 receives “Email sent successfully”
- Client picks option 4 and terminates connection .
- Server prints termination message



```
cheemar9@cc5-212-16:~/ProjectG3/server$ python3 server.py
The server is ready to accept connections
The received client information: client10 is invalid (Connection Terminated).
Connection Accepted and Symmetric Key Generated for client: client1
An email from client1 is sent to ['client2', 'client3'] has a content length of 20
Terminating connection with client1
Connection Accepted and Symmetric Key Generated for client: client3
Terminating connection with client3
Connection Accepted and Symmetric Key Generated for client: client2
Connection Accepted and Symmetric Key Generated for client: client5
Connection Accepted and Symmetric Key Generated for client: client4

cheemar9@cc5-212-11:~/ProjectG3/client$ python3 client.py
Enter server name: cc5-212-16.macewan.ca
Enter your username: client3
Enter your password: password3
Authentication successful.

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 2
Index From      DateTime           Title
1 client1  2024-06-16 17:38:16 ProjectTEST

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 3
Enter email index to view: 1
From: client1
To: client2, client3
Time and Date Received: 2024-06-16 17:38:16
Title: ProjectTEST
Content Length: 20
Contents:
this is project test

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 4
Connection terminated.
```

Fig. 4

Test 3 - View inbox and View email (Fig. 4)

- client 3 connects to server
- They select option 2 to view inbox list
- Inbox is displayed
- They select option 3 and input index 1 to view first email
- ProjectTEST email details and contents are displayed to client3
- Figure 5 shows client2 with same email that was sent earlier

```
-- ssh -l cheemar9 cc5-212-11  ...ssh -l cheemar9 cc5-212-10  ...h -l cheemar9 cc5-212-09
this is project test

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 4
Connection terminated.
cheemar9@cc5-212-11:~/ProjectG3/client$ python3 client.py
Enter server name: cc5-212-16.macewan.ca
Enter your username: client2
Enter your password: password2
Authentication successful.

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 2
Index From      DateTime          Title
1 client1  2024-06-16 17:38:16  ProjectTEST

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 3
Enter email index to view: 1
From: client1
To: client2, client3
Time and Date Received: 2024-06-16 17:38:16
Title: ProjectTEST
Content Length: 20
Contents:
this is project test

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: █
```

Fig .5

```
cheemar9@cc5-212-16:~/ProjectG3/server$ python3 server.py
The server is ready to accept connections
The received client information: client10 is invalid (Connection Terminated).
Connection Accepted and Symmetric Key Generated for client: client1
An email from client1 is sent to ['client2', 'client3'] has a content length of 20
Terminating connection with client1
Connection Accepted and Symmetric Key Generated for client: client3
Terminating connection with client3
Connection Accepted and Symmetric Key Generated for client: client2
Connection Accepted and Symmetric Key Generated for client: client5
Connection Accepted and Symmetric Key Generated for client: client4
An email from client5 is sent to ['client1'] has a content length of 26
```

Fig. 6

Test 4 - Multiple Users

- Figure 6 displays server side with multiple user connected at once from different lab machines shown in Figure 5 as the terminal tabs

- Unit testing
 - Several basic tests for the server that can be run without a client (utilizes a ‘dummy’ client)

Attack

- MITM - ‘Man/Woman-in-the-middle’ attack:
 - An attacker could intercept the messages between the client and the server and alter the messages.

Enhanced Protocol

Adding a hash function to the packet that is used in our encryption and cannot be duplicated by the attacker will help prevent an attack as described:

- Hash Function: To ensure the integrity of the messages, the protocol uses a hash function. When sending a message, the sender (either the client or the server) creates a hash of the message and sends this hash along with the message. The receiver creates a hash of the received message and compares it with the received hash. If the two hashes match, the message is verified to be untampered. This ensures that even if an attacker manages to intercept and alter the message, the receiver will be able to detect the tampering.
- Here’s a simplified example of how this works:
 - The sender creates a message.
 - The sender creates a hash of the message using the hash function.
 - The sender sends the message and the hash to the receiver.
 - The receiver receives the message and the hash.
 - The receiver creates a hash of the received message using the same hash function.
 - The receiver compares the received hash with the created hash. If they match, the message is verified.

Testing Server_enhanced.py and Client_enhanced.py

```
Rimis-MacBook-Pro:client rimicheema$ python3 client_enhanced.py
Enter server name: localhost
Enter your username: client1
Enter your password: password1
Authentication successful.

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 4
Connection terminated.
Rimis-MacBook-Pro:client rimicheema$ python3 client_enhanced.py
Enter server name: localhost
Enter your username: client1
Enter your password: password1
Authentication successful.

signature made with hashing the signing with client private key = b'.b\x89Y\xaf\x13\x0e)
MuH#\xe8\xa7\xc2\xe24 z\xda8\x8aA[\xa8c+\x9c\xfa\x05!"\xc7\x11:\x83[\x120aq\xef\x08\x80?
\x9f\x84P\x9f9\xbc\x07\x7f\x95\x1e\xf2\xe5\xfe3\xe2\xa9\x9d9\x80N\x7f\x7bdcWfw\xab2
\x8e\xeb\x06[F\x1e\xa8\|\x93\xabf\xda8\x1e\x14y\x93m\x05+\xf0\x03r\xabn\x0f!*:\|\x9d5
\xda6\xda7\x0b\x7f\xfa\xeb3\x08\xda5\x04+\xafh\x0a\x08\x0ff\x00\x17\x08\x05\x97\x99\x0b\x0f
2\x0e\x08f\x10\xda8\x064\x09\x1d\x09a\x19\x0b5\xac[\x0f\x04\x0b\x0c\xda\x05\x82I\x0c\x94\x
fdv\x06\xabfR\xeb-\x1a\x0e\x01\x18\x1f2\x0b\x0fda\t\x1f1\xef_\x02\x0b3\x066\x0b7\x0b4\x176v\x0
5/\xc7\|\x04\x26Z\xda3\x04\x80\x85\xde\x13\x00\x03\x0f\x0a\xcc\x00\x04%\xaa0\x00-\xaa7\x
dd\x1d\x86Pb(\xec\x86n\x83\xda4\xda8\xfa8\xda\x945m-\xa3\xad\x0c6+\xb9\xda1.\x1bY\x94\x8b\x8
e\x03\xdeq\x8c\xee[y\x8e\x0f'

Select the operation:
1) Create and send an email
2) Display the inbox list
3) Display the email contents
4) Terminate the connection

choice: 4
Connection terminated.
Rimis-MacBook-Pro:client rimicheema$
```

```
Rimis-MacBook-Pro:server rimicheema$ python3 server_enhanced.py
Server: Error in server socket binding: [Errno 48] Address already in use
Rimis-MacBook-Pro:server rimicheema$ python3 server_enhanced.py
The server is ready to accept connections
Connection Accepted and Symmetric Key Generated for client: client1
encrypted_ok_message = b'!\xd8\xe7\xd0\xbd}F\x06\xdc\xca\xde\xed\x90f\x0f1\xba'
sign = b'b'9\xf70\xd5\x83\xa3\xfa\xda94\x81\xda\x04\x01\x08\x0bfj\x99\x920\xbdy^\x07\xca\x06
\x9b\x0aX\x06\x8d\xfb'\x91s\xa8\xa6\x0e\x06\x97\xac\x965\x8bP-e\xcb\x124\x08\xfb\xda0dv\
x99Pa:\x9e8\x05\x0e\x02K\x0b\x0a9y,s\x03\x0a0\x0a=Q\x0e\x110\x0f\x03v\x0c7\x084\x0f0P\x80\x
f2\x04+\x98\x0b\x0a\x0a\x16\x93\x0f1\x0f3\x0f0\x01\x0f0\x16\x0a2a\x05\x160\x13\x02\x010\x9b\x
9d\x89n\x0b8;\x04X\x0a-\x11U5\x0a8q\x0eK\x0b\x06\x05\x8d\x0edj\x984M\x0f1\x0b1+\x0a\x0a\x0d\x
ff\x93\xda9q\x06\xef8y\x9b\x0cfy\x91\x19\xda9\x08d)V'\x13\x0f0\x1ft/\xb2\x0b0\x08\x0a\x06\x1e\x
180\x17\x0a2b,\x01,9\x0c9\xda7\x09f\x0fda\x0f8I\x0c32\x089E\x13\x08dT;\x05\x162\x0b\x05\x0a0l-\x09C0
\x0f4\x18\x0cb\x81V\x05\x057\x0a0\xda3\xda5\x0acpJ5m\x90}\x0d1K\x0e\x09\x0c0\x0b5\xfb\x120b0\x0
5\x0b\x15Y\x0a\x050\x01\x0fda\x0b0\x0bfb"
ok = OK
Terminating connection with client1
Connection Accepted and Symmetric Key Generated for client: client1
encrypted_ok_message = b'\xb0\x10\x0a9\x0b2\xe9w-\xf0\x9f\x0a1\x17\x0c81\x11e'
sign = b'b'.b\x89Y\xaf\x13\x0e)MuH#\xe8\xa7\xc2\xe24 z\xda8\x8aA[\xa8c+\x9c\xfa\x05!"\xc7\x
11:\x83[\x120aq\xef\x08\x80? \x9f\x84P\x9f9\xbc\x07\x7f\x95\x1e\xf2\xe5\xfe3\xe2\xa9\x9d9\x
da7\x0b3\x80N\x7f\x0bdcWfw\xab2\x8e\xeb\x06[F\x1e\xa8\|\x93\xabf\xda8\x1e\x14y\x93m\x05+\xf0\x
03r\xabn\x0f!*:\|\x9d5\xda6\xda7\x0b\x7f\xfa\xeb3\x08\xda5\x04+\xafh\x0a\x08\x0ff\x00\x17\x08\x05\x97\x99\x0b\x0f
2\x0e\x08f\x10\xda8\x064\x09\x1d\x09a\x19\x0b5\xac[\x0f\x04\x0b\x0c\xda\x05\x82I\x0c\x94\x
fdv\x06\xabfR\xeb-\x1a\x0e\x01\x18\x1f2\x0b\x0fda\t\x1f1\xef_\x02\x0b3\x066\x0b7\x0b4\x176v\x0
5/\xc7\|\x04\x26Z\xda3\x04\x80\x85\xde\x13\x00\x03\x0f\x0a\xcc\x00\x04%\xaa0\x00-\xaa7\x
dd\x1d\x86Pb(\xec\x86n\x83\xda4\xda8\xfa8\xda\x945m-\xa3\xad\x0c6+\xb9\xda1.\x1bY\x94\x8b\x8
e\x03\xdeq\x8c\xee[y\x8e\x0f'
ok = OK
Terminating connection with client1
```

Fig. 7

Client after passing authentication server send aes key to client, client will hash this using and then sign it using its private key and send to server along with its signature, then the server is waiting for the the clients OK response, it will receive the clients signature and encrypted OK message and hash the received message and decrypt it using the client public key and obtain the hash then match it with the signature to add one more step to the authentication process.

Resources

[Crypto.Cipher.AES](#)

- Provides AES encryption and decryption
- In this project, used for encrypted and secure communication between client and server

[Crypto.PublicKey.RSA](#)

- Provides RSA public key cryptography
- In this project, used for generate RSA key pairs and authentication

[Crypto.Random.get_random_bytes](#)

- Provides a way to generate random bytes
- In this project, used for generating AES symmetric key

[Crypto.Util.Padding](#)

- Provides utilities for padding and unpadding data
- In this project, used for padding and unpadding plaintext messages

[datetime](#)

- Provides supplies for dates and times
- In this project, used for recording timestamps for emails

[json](#)

- Provides methods for working with JSON data
- In this project, used for data to and from converting JSON format

[os](#)

- Provides using operating system functionality
- In this project, used for handling files and directory operations

[socket](#)

- Provides access to BSD socket interface, allowing for communication between machines over the network
- In this project, used for establishing connections between the server and client

SYS

- Provides access to system specific parameters and functions
- In this project, used for handling command-line arguments and exiting program